

Everything is Big-Crytohack

cryptografi

Analisis

deskripsi chall:

We have a supercomputer at work, so I've made sure my encryption is secure by picking massive numbers!

Challenge files:

- [source.py](#)
- [output.txt](#)

isi output.txt:

N =

```
0xb8af3d3afb893a602de4afe2a29d7615075d1e570f8bad8ebbe9b5b9076594cf06b6e7b30905b6420e9
50043380ea746f0a14dae34469aa723e946e484a58bcd92d1039105871ffd63ffe64534b7d7f8d84b4a56
9723f7a833e6daf5e182d658655f739a4e37bd9f4a44aff6ca0255cda5313c3048f56eed5b21dc8d88bf5a
8f8379eac83d8523e484fa6ae8dbcb239e65d3777829a6903d779cd2498b255fcf275e5f49471f3599243
5ee7cade98c8e82a8beb5ce1749349caa16759afc4e799edb12d299374d748a9e3c82e1cc983cdf9daec0a
2739dadcc0982c1e7e492139cbff18c5d44529407edfd8e75743d2f51ce2b58573fea6fbd4fe25154b9964
d
```

e =

```
0x9ab58dbc8049b574c361573955f08ea69f97ecf37400f9626d8f5ac55ca087165ce5e1f459ef6fa5f158c
c8e75cb400a7473e89dd38922ead221b33bc33d6d716fb0e4e127b0fc18a197daf856a7062b49fba7a86e
3a138956af04f481b7a7d481994aeebc2672e500f3f6d8c581268c2cfad4845158f79c2ef28f242f4fa8f6e
573b8723a752d96169c9d885ada59cdeb6dbe932de86a019a7e8fc8aeb07748cfb272bd36d94fe833512
52187c2e0bc58bb7a0a0af154b63397e6c68af4314601e29b07caed301b6831cf34caa579eb42a8c8bf69
898d04b495174b5d7de0f20cf2b8fc55ed35c6ad157d3e7009f16d6b61786ee40583850e67af13e9d25be
3
```

c =

```
0x3f984ff5244f1836ed69361f29905ca1ae6b3dcf249133c398d7762f5e277919174694293989144c9d2
5e940d2f66058b2289c75d1b8d0729f9a7c4564404a5fd4313675f85f31b47156068878e236c5635156b
0fa21e24346c2041ae42423078577a1413f41375a4d49296ab17910ae214b45155c4570f95ca874ccae9f
a80433a1ab453cbb28d780c2f1f4dc7071c93aff3924d76c5b4068a0371dff82531313f281a8acadaa2bd5
078d3ddcefc981f37ff9b8b14c7d9bflaccffe7857160982a2c7d9ee01d3e82265eec9c7401ecc7f02581f
d0d912684f42d1b71df87a1ca51515aab4e58fab4da96e154ea6cdfb573a71d81b2ea4a080a1066e1bc34
```

74

sc chall:

```
#!/usr/bin/env python3

from Crypto.Util.number import getPrime, bytes_to_long

FLAG = b"crypto{????????????????????}"

m = bytes_to_long(FLAG)

def get_huge_RSA():
    p = getPrime(1024)
    q = getPrime(1024)
    N = p*q
    phi = (p-1)*(q-1)
    while True:
        d = getPrime(256)
        e = pow(d,-1,phi)
        if e.bit_length() == N.bit_length():
            break
    return N,e

N, e = get_huge_RSA()
c = pow(m, e, N)

print(f'N = {hex(N)}')
print(f'e = {hex(e)}')
print(f'c = {hex(c)}')
```

dari source code kita di diberitahu bahwa ini adalah chall RSA cryptografi,dimana e di buat sangat besar(sama dengan panjang bit dari n) dan d dibuat kecil.

Solusi

karena nilai bit d kecil,maka kerentanan di chall ini adalah di sana.Mengapa?karena nilai e yang panjang bitnya sama dengan n dapat membuat nilai d yang kita gunakan untuk medekripsi chipher text akan berukuran kecil(di source code chall panjang d adalah 256bit) dimana membuat d rentan dengan wiener's attack.cara kerja nya adalah n dan e akan di pecah menjadi pecahan berlanjut dengan membagi keduanya (e/n).lalu dari hasil pembagian itu nilai d ditembak satu persatu,karena panjang bit dari d kecil(256 bit) maka d akan tertebak dalam beberapa detik.

kode solver:

```
import ContinuedFractions, Arithmetic
from Crypto.Util.number import long_to_bytes
def hack_RSA(e,n):
    '''
    Finds d knowing (e,n)
    applying the Wiener continued fraction attack
    '''
    _, convergents = ContinuedFractions.rational_to_contfrac(e, n)

    for (k,d) in convergents:

        #check if d is actually the key
        if k!=0 and (e*d-1)%k == 0:
            phi = (e*d-1)//k
            s = n - phi + 1
            # check if the equation x^2 - s*x + n = 0
            # has integer roots
            discr = s*s - 4*n
            if(discr>=0):
                t = Arithmetic.is_perfect_square(discr)
                if t!=-1 and (s+t)%2==0:
                    print("Hacked!")
                    return d

n =
int('0xb8af3d3afb893a602de4afe2a29d7615075d1e570f8bad8ebbe9b5b9076594cf
06b6e7b30905b6420e950043380ea746f0a14dae34469aa723e946e484a58bcd92d1039
105871ffd63ffe64534b7d7f8d84b4a569723f7a833e6daf5e182d658655f739a4e37bd
9f4a44aff6ca0255cda5313c3048f56eed5b21dc8d88bf5a8f8379eac83d8523e484fa6
ae8dbcb239e65d3777829a6903d779cd2498b255fcf275e5f49471f35992435ee7cade9
8c8e82a8beb5ce1749349caa16759afc4e799edb12d299374d748a9e3c82e1cc983cdf9
daec0a2739dadcc0982c1e7e492139cbff18c5d44529407edfd8e75743d2f51ce2b5857
3fea6fbd4fe25154b9964d',16)
e =
int('0x9ab58dbc8049b574c361573955f08ea69f97ecf37400f9626d8f5ac55ca08716
5ce5e1f459ef6fa5f158cc8e75cb400a7473e89dd38922ead221b33bc33d6d716fb0e4e
127b0fc18a197daf856a7062b49fba7a86e3a138956af04f481b7a7d481994aeebc2672
e500f3f6d8c581268c2cfad4845158f79c2ef28f242f4fa8f6e573b8723a752d96169c9
d885ada59cdeb6dbe932de86a019a7e8fc8aeb07748cfb272bd36d94fe83351252187c2
e0bc58bb7a0a0af154b63397e6c68af4314601e29b07caed301b6831cf34caa579eb42a
```

```

8c8bf69898d04b495174b5d7de0f20cf2b8fc55ed35c6ad157d3e7009f16d6b61786ee4
0583850e67af13e9d25be3',16)

c =
int('0x3f984ff5244f1836ed69361f29905ca1ae6b3dcf249133c398d7762f5e277919
174694293989144c9d25e940d2f66058b2289c75d1b8d0729f9a7c4564404a5fd431367
5f85f31b47156068878e236c5635156b0fa21e24346c2041ae42423078577a1413f4137
5a4d49296ab17910ae214b45155c4570f95ca874ccae9fa80433a1ab453cbb28d780c2f
1f4dc7071c93aff3924d76c5b4068a0371dff82531313f281a8acadaa2bd5078d3ddcef
cb981f37ff9b8b14c7d9bflaccffe7857160982a2c7d9ee01d3e82265eec9c7401ecc7f
02581fd0d912684f42d1b71df87a1ca51515aab4e58fab4da96e154ea6cdfb573a71d81
b2ea4a080a1066e1bc3474',16)

# d = hack_RSA(e,n)
# print(d)

d =
79434351637397000170240219617391501050474168352481334243649813782018808
904459

m = pow(c,d,n)
print(long_to_bytes(m))

```

kode module ContinuedFractions:

```

'''
Created on Dec 14, 2011

@author: pablocelayes

'''
# Types
CFListT = list[int] # CF coefficients
CVListT = list[tuple[int, int]] # Convergents at each coefficient
level

def rational_to_contfrac(x: int, y: int) -> tuple[CFListT, CVListT]:
    """
    Converts a rational x/y fraction into
    a list of partial coefficients [a0, ..., an], and
    a list of convergents at each coefficient level [(n0, d0), (n1,
    d1), ...]
    """

```

The algorithm of computing the convergents from left to right is available

in Section 9.1 of

<https://r-knott.surrey.ac.uk/Fibonacci/cfINTRO.html#CFtofract>

Args:

x (int): numerator of the given rational number

y (int): denominator of the given rational number

Returns:

tuple[CFListT, CVListT]: a tuple of coefficients and convergents at each coefficient level

"""

a = x // y

cflist = [a]

cvlist = [(a, 1)]

ppn, ppd = 1, 0 # pre-pre numerator and denominator of convergent

pn, pd = a, 1 # pre numerator and denominator of convergent

while a * y != x:

 x, y = y, x - a * y

 a = x // y

 cflist.append(a)

 cn, cd = a * pn + ppn, a * pd + ppd

 cvlist.append((cn, cd))

 ppn, ppd = pn, pd

 pn, pd = cn, cd

return cflist, cvlist

kode module Arithmetic:

'''

Created on Dec 22, 2011

@author: pablocelayes

'''

def egcd(a,b):

 '''

 Extended Euclidean Algorithm

 returns x, y, gcd(a,b) such that ax + by = gcd(a,b)

 '''

 u, u1 = 1, 0

```

v, v1 = 0, 1
while b:
    q = a // b
    u, u1 = u1, u - q * u1
    v, v1 = v1, v - q * v1
    a, b = b, a - q * b
return u, v, a

def gcd(a,b):
    '''
    2.8 times faster than egcd(a,b)[2]
    '''
    a,b=(b,a) if a<b else (a,b)
    while b:
        a,b=b,a%b
    return a

def modInverse(e,n):
    '''
    d such that de = 1 (mod n)
    e must be coprime to n
    this is assumed to be true
    '''
    return egcd(e,n)[0]%n

def totient(p,q):
    '''
    Calculates the totient of pq
    '''
    return (p-1)*(q-1)

def bitlength(x):
    '''
    Calculates the bitlength of x
    '''
    assert x >= 0
    n = 0
    while x > 0:
        n = n+1
        x = x>>1
    return n

```

```

def isqrt(n):
    '''
    Calculates the integer square root
    for arbitrary large nonnegative integers
    '''
    if n < 0:
        raise ValueError('square root not defined for negative
numbers')

    if n == 0:
        return 0
    a, b = divmod(bitlength(n), 2)
    x = 2**(a+b)
    while True:
        y = (x + n//x)//2
        if y >= x:
            return x
        x = y

def is_perfect_square(n):
    '''
    If n is a perfect square it returns sqrt(n),

    otherwise returns -1
    '''
    h = n & 0xF; #last hexadecimal "digit"

    if h > 9:
        return -1 # return immediately in 6 cases out of 16.

    # Take advantage of Boolean short-circuit evaluation
    if ( h != 2 and h != 3 and h != 5 and h != 6 and h != 7 and h != 8
):
        # take square root if you must
        t = isqrt(n)
        if t*t == n:
            return t
        else:
            return -1

    return -1

```